

TenderWatch

Frontend: Vue 3 SPA для тендеров, подписок и уведомлений

Техническое задание на фронтенд

Стек: Vue 3 (Composition API), Vite, Pinia, Vue Router, TypeScript

Версия документа: 1.0 · Дополняет основное ТЗ TenderWatch (backend)

Дата: 18 июня 2026

Содержание

1. Цель и назначение
2. стек и архитектура приложения
3. Карта экранов
4. Модуль «Тендеры»
5. Модуль «Подписки»
6. Модуль «Лог уведомлений»
7. Слой API и интеграция с бэкендом
8. Состояние приложения (Pinia stores)
9. UX-детали: загрузка, ошибки, пустые состояния
10. Визуальный стиль
11. План по дням (неделя 4)
12. Критерии готовности (Definition of Done)

1. Цель и назначение

Фронтенд TenderWatch — это Vue 3 SPA, демонстрирующее полный пользовательский цикл сервиса: просмотр и фильтрация тендеров, настройка подписок с критериями отбора, просмотр лога доставленных уведомлений. Документ дополняет основное ТЗ на backend (Yii3) и не повторяет архитектурные решения бэкенда — только то, что касается клиентской части и её интеграции с REST API.

Фронтенд не является фокусом проекта — он нужен, чтобы дать собеседнику и интервьюеру возможность увидеть продукт целиком за 2–3 минуты демо, без необходимости дёргать API через Postman.

1.1 Что показывает демо

1. Список актуальных тендеров с фильтрами (категория, бюджет, регион, поиск по тексту).
2. Создание подписки: выбор критериев + каналов доставки (email/Telegram).
3. Лог уведомлений: какие тендеры подошли под какие подписки и были отправлены.

1.2 Явно вне рамок

- Полноценная регистрация/восстановление пароля — упрощённая форма логина с тест-пользователем или magic-link.
- Адаптивная мобильная версия как отдельная задача — достаточно базовой адаптивности (flex/grid), не mobile-first дизайн.
- Real-time обновления через WebSocket — список обновляется по запросу/поллингу, не критично для демо.

2. Стек и архитектура приложения

Слой	Технология	Назначение
Core	Vue 3 (Composition API), TypeScript	Базовый фреймворк, типизация контрактов API
Build	Vite	Dev-сервер, сборка
Роутинг	Vue Router 4	Маршруты, ленивая загрузка страниц
Состояние	Pinia	Stores по доменам: tenders, subscriptions, notifications, auth
HTTP	ofetch или axios	Обёртка над REST API с интерцепторами (auth-токен, обработка ошибок)
Формы/валидация	VeeValidate + zod (или yup)	Валидация формы подписки на клиенте
UI-кит	Без тяжёлого фреймворка — кастомные компоненты + Tailwind CSS	Полный контроль над видом, минимум зависимостей
Тесты	Vitest + Vue Test Utils	Unit-тесты на stores и ключевые компоненты

2.1 Структура каталогов

```

src/
├─ api/                → клиенты для каждого ресурса (tenders.ts,
subscriptions.ts, ...)
├─ stores/             → Pinia stores по доменам
├─ views/              → страницы, привязанные к роутам
├─ components/         → переиспользуемые UI-компоненты
│   └─ tenders/
│   └─ subscriptions/
│   └─ notifications/
├─ composables/        → useTenderFilters, usePagination, useToast и т.д.
├─ types/              → TypeScript-типы, синхронизированные с OpenAPI-спекой
бэкенда
└─ router/

```

Типы в `src/types/` по возможности генерируются из OpenAPI-спеки бэкенда (`openapi-typescript`), чтобы контракт фронта и бэкенда не расходился вручную — хороший повод показать на интервью осознанный подход к типобезопасности на границе систем.

2.2 Аутентификация

- Упрощённая модель: логин по email без пароля (magic-link через email) либо тестовый логин с заранее заданным пользователем — выбирается на этапе разработки исходя из оставшегося времени.
- Токен (Bearer) хранится в памяти приложения (Pinia store) + дублируется в `httpOnly cookie` на стороне бэкенда, если потребуется рефреш при перезагрузке страницы.

- Маршруты Subscriptions и Notifications защищены навигационным guard'ом Vue Router.

3. Карта экранов

Маршрут	Страница	Доступ
/	Список тендеров (главная)	Публичный
/tenders/:id	Детали тендера	Публичный
/login	Вход	Публичный
/subscriptions	Список подписок пользователя	Авторизован
/subscriptions/new	Создание подписки	Авторизован
/subscriptions/:id/edit	Редактирование подписки	Авторизован
/notifications	Лог уведомлений	Авторизован

3.1 Навигация

Верхний навбар: логотип/название, ссылки «Тендеры», «Подписки», «Уведомления» (две последние видны только авторизованным), индикатор пользователя и кнопка выхода справа.

4. Модуль «Тендеры»

4.1 Список тендеров

- Карточка тендера: заголовок, категория (tag), бюджет, регион, дедлайн (с относительным временем — «осталось 3 дня»), статус.
- Панель фильтров слева или сверху: категория (select), диапазон бюджета (два поля от/до), регион (select), полнотекстовый поиск (debounce 400мс).
- Пагинация — курсорная или offset-based, синхронизирована с параметрами URL (чтобы ссылку можно было передать с конкретным фильтром).
- Сортировка: по дедлайну (по умолчанию), по бюджету, по дате публикации.

4.2 Детали тендера

- Полное описание, метаданные, ссылка на источник (если применимо).
- Блок «Тендер подходит под подписки» — если пользователь авторизован, показывает совпадение с его активными подписками (опционально, если останется время).

4.3 Состояния экрана

Состояние	Поведение UI
Загрузка	Skeleton-карточки (3–6 штук) вместо списка
Пусто (фильтр не дал результатов)	Иллюстрация/текст «Ничего не найдено» + кнопка сброса фильтров
Ошибка сети/сервера	Карточка ошибки с кнопкой «Повторить», без потери введённых фильтров
Частичная ошибка пагинации	Тост с ошибкой, текущая страница остаётся видимой

5. Модуль «Подписки»

5.1 Список подписок

- Таблица/список: название подписки, краткое описание критериев (например, «IT, бюджет 100k–1M, Москва»), активные каналы (иконки email/Telegram), переключатель isActive, действия (редактировать/удалить).
- Пустое состояние для нового пользователя: объяснение, как работает подписка + кнопка «Создать первую подписку».

5.2 Форма создания/редактирования подписки

- Название подписки (текстовое поле, опциональное — генерируется автоматически из критериев, если не указано).
- Категории — multi-select с возможностью поиска.
- Диапазон бюджета — два числовых поля с валидацией ($\min \leq \max$).
- Регионы — multi-select.
- Ключевые слова — поле с тегами (ввод через Enter/запятую).
- Каналы доставки — чекбоксы Email / Telegram, с подсказкой подключить Telegram-бота, если chat_id не привязан.
- Live-предпросмотр: количество тендеров за последние 30 дней, подходящих под текущие критерии (debounced запрос к API при изменении формы) — наглядно показывает пользователю ценность подписки до сохранения.

5.3 Валидация на клиенте

- Хотя бы один критерий должен быть указан (нельзя сохранить полностью пустую подписку).
- Хотя бы один канал доставки выбран.
- minBudget не больше maxBudget.
- Серверная валидация (422) маппится на конкретные поля формы через единый формат ошибок API.

6. Модуль «Лог уведомлений»

6.1 Список уведомлений

- Хронологический список: тендер (ссылка на детали), подписка, которая сработала, канал доставки, статус (отправлено/ошибка), время.
- Фильтр по подписке и по статусу (всё / успешно / ошибка).
- Для статуса «ошибка» — раскрывающаяся деталь с текстом ошибки (например, недоступен Telegram API).

6.2 Связь с другими модулями

Каждая запись лога ссылается на конкретный тендер (переход в /tenders/:id) и на подписку (переход в /subscriptions/:id/edit) — это завершает демонстрацию полного цикла: тендер → подписка → совпадение → уведомление, не выходя за пределы фронтенда.

7. Слой API и интеграция с бэкендом

Фронтенд работает с REST API, описанным в основном ТЗ (раздел «REST API»). Ниже — соответствие экранов и эндпоинтов.

Экран/действие	Эндпоинт
Список тендеров	GET /api/v1/tenders (query: category, region, budget, q, page)
Детали тендера	GET /api/v1/tenders/{id}
Список подписок	GET /api/v1/subscriptions
Создание подписки	POST /api/v1/subscriptions
Редактирование подписки	PATCH /api/v1/subscriptions/{id}
Удаление/деактивация подписки	DELETE /api/v1/subscriptions/{id}
Лог уведомлений	GET /api/v1/notifications (новый эндпоинт, см. ниже)
Предпросмотр подписки	GET /api/v1/subscriptions/preview?criteria=... (новый эндпоинт, см. ниже)

7.1 Дополнения к API бэкенда, нужные фронту

Эти два эндпоинта не были явно описаны в основном ТЗ и должны быть добавлены на бэкенде для поддержки фронтенда:

- GET /api/v1/notifications — список из таблицы notifications_log с фильтрами по subscription_id и status, постранично.
- GET /api/v1/subscriptions/preview — принимает критерии как query-параметры (без сохранения подписки) и возвращает count + несколько примеров подходящих тендеров за последние N дней. Используется для live-предпросмотра в форме подписки.

7.2 Обработка ошибок API

- 401 — редирект на /login, очистка стейта пользователя.
- 403 — страница/тост «Доступ запрещён».
- 422 — маппинг ошибок по полям в форму.
- 429 — тост «Слишком много запросов, попробуйте позже» с учётом Retry-After, если бэкенд его передаёт.
- 5xx / network error — глобальный тост + возможность повторить последнее действие.

8. Состояние приложения (Pinia stores)

Store	Состояние	Действия
auth	user, token, isAuthenticated	login, logout, restoreSession
tenders	items, filters, pagination, isLoading	fetchList, fetchOne, setFilter, resetFilters
subscriptions	items, isLoading	fetchList, create, update, remove, preview
notifications	items, filters, pagination, isLoading	fetchList, setFilter
ui	toasts[], globalError	pushToast, dismissToast

Каждый store инкапсулирует обращение к своему API-клиенту (`src/api/*.ts`) — компоненты не вызывают `fetch/axios` напрямую, только через actions store. Это сознательное архитектурное решение, зеркальное Repository-паттерну на бэкенде: компонент не знает, откуда взялись данные.

9. UX-детали: загрузка, ошибки, пустые состояния

- Skeleton-плейсхолдеры вместо спиннеров везде, где есть список/карточка — ощущение скорости важнее технической честности.
- Тосты — единый компонент в правом верхнем углу, авто-скрытие через 4 секунды для успеха, требуют закрытия для ошибок.
- Оптимистичные обновления для переключателя isActive подписки — мгновенный отклик UI, откат при ошибке сервера.
- Дебаунс 300–400мс на все текстовые фильтры и поля предпросмотра, чтобы не спамить API.
- Подтверждение перед удалением подписки — модальное окно, а не window.confirm.

10. Визуальный стиль

- Tailwind CSS, нейтральная палитра с одним акцентным цветом (синий/индиго) для primary-действий и статусов «совпадение/успех».
- Шрифт системный (Inter/системный sans-serif стек) — не грузить веб-шрифт ради демо-проекта.
- Карточки с мягкой тенью и небольшим радиусом скругления, плотная, но читаемая типографика — ориентир: стиль панелей Linear/Vercel Dashboard, не перегруженный.
- Тёмная тема — опционально, если останется время после основного функционала.

11. План по дням (неделя 4 общего плана проекта)

Фронтенд разрабатывается в рамках 4-й недели общего плана TenderWatch (см. основное ТЗ, раздел «План по неделям»), параллельно с полировкой бэкенда. Ориентировочная раскладка:

День 1 — Каркас

- Инициализация Vite + Vue 3 + TS + Tailwind, роутинг, базовый layout с навбаром.
- Настройка API-клиента с интерцепторами авторизации и единой обработкой ошибок.
- auth store + страница логина (упрощённая).

День 2 — Тендеры

- Список тендеров: карточки, фильтры, пагинация, синхронизация с URL.
- Skeleton/empty/error состояния.
- Страница деталей тендера.

День 3 — Подписки

- Список подписок, форма создания/редактирования с валидацией.
- Интеграция live-предпросмотра (после готовности /subscriptions/preview на бэкенде).
- Удаление/деактивация с подтверждением.

День 4 — Уведомления и доводка

- Лог уведомлений с фильтрами и переходами на тендер/подписку.
- Тосты, оптимистичные обновления, финальная проверка всех состояний загрузки/ошибок.
- Базовые unit-тесты на stores (Vitest).
- Сборка production-бандла, проверка перед деплоем вместе с бэкендом.

12. Критерии готовности (Definition of Done)

- Все три модуля (тендеры, подписки, уведомления) работают end-to-end на реальном API бэкенда, без моков.
 - Каждый список имеет явные состояния: загрузка, пусто, ошибка — без "мигания" пустым экраном.
 - Форма подписки валидируется на клиенте и корректно показывает серверные ошибки 422 по полям.
 - Полный пользовательский путь (тендер → подписка → уведомление в логе) проходим за одну демо-сессию без перезагрузки страницы.
 - TypeScript строгий режим включён, типы синхронизированы с контрактом API.
 - npm run build собирается без ошибок и warning'ов.
-

Документ дополняет основное ТЗ TenderWatch (backend). Раздел 7.1 фиксирует два эндпоинта, которые необходимо добавить на бэкенде для полноценной работы фронтенда.